

Abaixo está um exemplo **completo e didático** de uma API REST usando Spring Boot com banco em memória H2 Database para gerenciar **contas bancárias (titular e saldo)** com CRUD.

Estrutura do projeto

```
conta-api/  
├── controller/  
├── service/  
├── repository/  
├── model/  
└── ContaApiApplication.java
```

1. Dependências (pom.xml)

```
<dependencies>  
    <!-- Web -->  
    <dependency>  
        <groupId>org.springframework.boot</groupId>  
        <artifactId>spring-boot-starter-web</artifactId>  
    </dependency>  
  
    <!-- JPA -->  
    <dependency>  
        <groupId>org.springframework.boot</groupId>  
        <artifactId>spring-boot-starter-data-jpa</artifactId>  
    </dependency>  
  
    <!-- H2 -->  
    <dependency>  
        <groupId>com.h2database</groupId>  
        <artifactId>h2</artifactId>  
        <scope>runtime</scope>  
    </dependency>  
  
    <!-- Lombok (opcional) -->
```

```
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <optional>true</optional>
</dependency>
</dependencies>
```

2. Configuração (application.properties)

```
spring.datasource.url=jdbc:h2:mem:contadb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
```

```
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=update
```

```
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
```

👉 Acesse o console:

<http://localhost:8080/h2-console>

3. Entidade (Conta)

```
package com.example.contaapi.model;
```

```
import jakarta.persistence.*;
import java.math.BigDecimal;
```

```
@Entity
public class Conta {
```

```
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```
private String titular;

private BigDecimal saldo;

public Conta() {}

public Conta(String titular, BigDecimal saldo) {
    this.titular = titular;
    this.saldo = saldo;
}

public Long getId() { return id; }

public String getTitular() { return titular; }

public void setTitular(String titular) {
    this.titular = titular;
}

public BigDecimal getSaldo() { return saldo; }

public void setSaldo(BigDecimal saldo) {
    this.saldo = saldo;
}
}
```

4. Repository

```
package com.example.contaapi.repository;

import com.example.contaapi.model.Conta;
import org.springframework.data.jpa.repository.JpaRepository;

public interface ContaRepository extends JpaRepository<Conta, Long> {
}
```

5. Service

```
package com.example.contaapi.service;

import com.example.contaapi.model.Conta;
import com.example.contaapi.repository.ContaRepository;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class ContaService {

    private final ContaRepository repository;

    public ContaService(ContaRepository repository) {
        this.repository = repository;
    }

    public Conta salvar(Conta conta) {
        return repository.save(conta);
    }

    public List<Conta> listar() {
        return repository.findAll();
    }

    public Conta buscarPorId(Long id) {
        return repository.findById(id)
            .orElseThrow(() -> new RuntimeException("Conta não encontrada"));
    }

    public Conta atualizar(Long id, Conta contaAtualizada) {
        Conta conta = buscarPorId(id);
        conta.setTitular(contaAtualizada.getTitular());
        conta.setSaldo(contaAtualizada.getSaldo());
        return repository.save(conta);
    }
}
```

```
    }

    public void deletar(Long id) {
        repository.deleteById(id);
    }
}
```

6. Controller (API REST)

```
package com.example.contaapi.controller;

import com.example.contaapi.model.Conta;
import com.example.contaapi.service.ContaService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.net.URI;
import java.util.List;

@RestController
@RequestMapping("/contas")
public class ContaController {

    private final ContaService service;

    public ContaController(ContaService service) {
        this.service = service;
    }

    // CREATE → 201 Created
    @PostMapping
    public ResponseEntity<Conta> criar(@RequestBody Conta conta) {
        Conta novaConta = service.salvar(conta);

        return ResponseEntity
            .created(URI.create("/contas/" + novaConta.getId()))
            .body(novaConta);
    }
}
```

```

    }

    // READ (todos) → 200 OK
    @GetMapping
    public ResponseEntity<List<Conta>> listar() {
        List<Conta> contas = service.listar();
        return ResponseEntity.ok(contas);
    }

    // READ (por id) → 200 OK ou 404 Not Found
    @GetMapping("/{id}")
    public ResponseEntity<Conta> buscar(@PathVariable Long id) {
        try {
            Conta conta = service.buscarPorId(id);
            return ResponseEntity.ok(conta);
        } catch (RuntimeException e) {
            return ResponseEntity.notFound().build();
        }
    }

    // UPDATE → 200 OK ou 404 Not Found
    @PutMapping("/{id}")
    public ResponseEntity<Conta> atualizar(@PathVariable Long id,
                                           @RequestBody Conta conta) {
        try {
            Conta atualizada = service.atualizar(id, conta);
            return ResponseEntity.ok(atualizada);
        } catch (RuntimeException e) {
            return ResponseEntity.notFound().build();
        }
    }

    // DELETE → 204 No Content ou 404 Not Found
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deletar(@PathVariable Long id) {
        try {
            service.deletar(id);
            return ResponseEntity.noContent().build();
        }
    }

```

```
        } catch (RuntimeException e) {
            return ResponseEntity.notFound().build();
        }
    }
}
```

7. Classe principal

```
package com.example.contaapi;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class ContaApiApplication {

    public static void main(String[] args) {
        SpringApplication.run(ContaApiApplication.class, args);
    }
}
```

8. Testando a API

Criar conta

POST /contas

```
{
  "titular": "João",
  "saldo": 1000.00
}
```

Listar contas

GET /contas

Buscar por ID

```
GET /contas/1
```

Atualizar

```
PUT /contas/1
```

```
{  
  "titular": "João Silva",  
  "saldo": 2000.00  
}
```

Deletar

```
DELETE /contas/1
```

Mapeamento dos status usados

Operação	Status correto	Motivo
POST	201 Created	Recurso criado
GET	200 OK	Retorno com dados
GET (não encontrado)	404 Not Found	Recurso inexistente
PUT	200 OK	Atualização realizada
DELETE	204 No Content	Exclusão sem retorno